Munchkin: Reinforcement Learning for Dynamic Food-Delivery Dispatch

Anshit Agarwal · Namya Dhingra

GitHub: github.com/Anshit2005/Munchkin---RL-Based-Dynamic-Food-Delivery-Dispatch-System

Abstract— We present Munchkin, a minute-stepped food-delivery dispatch simulator and comparative study of three heuristics, Deep Q-Network (DQN), and Proximal Policy Optimization (PPO). A fixed 60-minute episode couples stochastic order arrivals, restaurant preparation, travel, deadlines, and finite driver capacity. The observation contains 1,849 features and the masked discrete action space contains 1,025 actions. Across 11 five-seed scenario configurations, heuristics remain highly competitive; learned policies show useful behavior in selected conditions but do not dominate consistently.

1. Problem and environment

Munchkin models online assignment of food orders to delivery partners. Each episode simulates one hour: one real-time second corresponds to one game minute. The map is 100×70 units, with 3–8 restaurants and 3–16 drivers during training. New orders follow a Poisson process with scenario rate $\lambda \in [1,8]$ orders/minute; half the configured rate is seeded as initial backlog. Preparation time is a clipped normal sample (mean 0.5, SD 0.17 min, bounds 0.15–1.0); estimates are log-normal noisy (σ=0.2). Deadlines equal estimated preparation plus nominal travel time and five minutes of slack. Traffic uses light/normal/heavy multipliers 1.2/1.0/0.7 with minute-level noise. Drivers move, wait for food, and deliver as time advances; unfinished orders incur a terminal penalty.

2. RL formulation

State space. At each minute the fixed-size observation $s_t \in R^{1849}$ concatenates 16 driver records (11 values each), 64 order records (10 each), a 16×64 driver–order ETA matrix, and 9 global features: 16(11)+64(10)+16(64)+9=1849. Driver features encode presence, position, availability/travel state, destination, remaining distance/time, and food-wait status. Order features encode presence, restaurant/customer coordinates, age, preparation remaining, restaurant load, route length, and deadline slack. Global values encode normalized time/horizon, dispatchable/pending/available counts, demand rate, restaurant count, overflow, and traffic ratio. Padding fixes dimensions; masks exclude absent or infeasible pairs.

Action space. Discrete dispatch driver i to order j, encoded 64i+j for i<16,j<64, or WAIT (index 1024): |A|=1,025. Only feasible pairs are enabled. Assignment consumes no simulated minute, allowing multiple dispatches before WAIT advances the clock.

Reward. Shared shaped reward per step:

$$r_t = 10D_t - 2W_t - 4L_t - 0.1M_t - I_t - 5F_t \cdot 1[t=T].$$

D = new deliveries; W = pending orders after movement; L = overdue pending; M = distance moved; I=min(available drivers, dispatchable unassigned orders) (idle-capacity cost); F = unfinished pending orders at the 60-minute cutoff. Game score is separately 10 points per delivery; it is not the shaped RL return.

3. Policies and training

Baselines are Nearest Driver, Oldest/Most Urgent (least deadline slack), and Minimum Estimated Delivery Time. DQN and PPO share state, reward, and action masks. DQN uses pairwise Q values, masked Double-DQN targets, uniform replay (capacity 20,000; batch 128), γ=0.99, learning rate 3e-4, warm-up 32 transitions, updates every four steps, target sync every 1,000 steps, and ε decay 1.0→0.05. PPO uses masked categorical actions, clipped objective ε=0.2, γ=0.99, GAE λ=0.95, 8 epochs, minibatches of 128, and learning rate 3e-4. PPO samples during training and uses deterministic masked argmax at evaluation. Both trained for 500 episodes over randomized demand, fleet, restaurant count, and traffic. Best checkpoints: DQN episode 175 (validation shaped return –3,949.3) and PPO episode 300 (–4,806.9). The sweep uses five matched seeds per cell (11 configurations; 275 policy-episodes total), each 60 minutes.

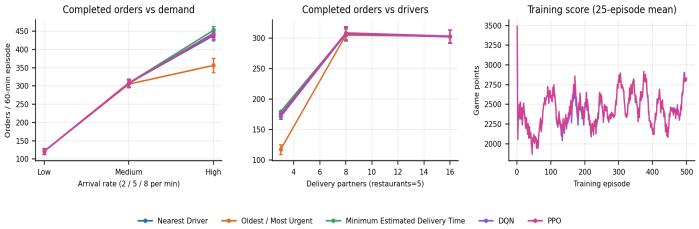


Fig. 1. Results across demand, driver capacity, and training. Error bars show across-seed SD for completed orders; training uses a 25-episode moving mean of game points.

4. Results and discussion

Table 1. Mean completed orders / game points by demand under normal traffic (five seeds).

Policy	Low	Medium	High
Nearest	121.2 / 1212	307.4 / 3074	437.8 / 4378
Urgent	121.0 / 1210	305.0 / 3050	356.2 / 3562
Min ETA	121.2 / 1212	307.8 / 3078	452.4 / 4524
DQN	120.6 / 1206	306.8 / 3068	444.4 / 4444
PPO	121.2 / 1212	308.6 / 3086	440.8 / 4408

At low and medium demand, methods are close; PPO leads medium-demand points (3,086), with Minimum ETA at 3,078. At high demand, Minimum ETA completes most orders (452.4); DQN completes 444.4 but has the best mean shaped return (–1,184 versus –1,681 for Minimum ETA). Thus game score and shaped reward rank policies differently. With medium demand and three drivers, Minimum ETA completes 179.4 versus DQN 171.0 and PPO 175.0. In the severe high-demand/heavy-traffic case with three restaurants and three drivers, Minimum ETA completes 177.6, PPO 144.6, and DQN 100.4. These results do not show RL universally outperforming heuristics; the constrained-fleet case exposes a weakness of the learned dispatchers.

Conclusion. Munchkin provides a reproducible benchmark with explicit state/action masks and shared scenario dynamics. Current learned policies are competitive in selected regimes but not uniformly superior. Future work should target constrained-fleet credit assignment, broader reward/hyperparameter sweeps, and more seeds before statistical conclusions.

References

[1] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, 518, 529–533, 2015. doi:10.1038/nature14236.
[2] J. Schulman et al., “Proximal Policy Optimization Algorithms,” arXiv:1707.06347, 2017.
[3] Munchkin project repository and simulator implementation, GitHub, 2026.